بسم الله الرحمن الرحیم



موضوع: تکلیف سری سوم

درس هوش مصنوعی مقدماتی

نام و نام خانوادگی: محمدمهدی آزادی

شماره دانشجویی: 40214623

نام استاد: استاد تشنه‌لب

تکلیف درس هوش مصنوعی مقدماتی

1404-1405

فهرست

Table of Contents

مقدمه 1

سوال اول (بخش اول) 2

سوال دوم (بخش اول) 4

سوال سوم (بخش اول) 6

سوال چهارم (بخش دوم) 9

الف) **Data Preprocessing and Preparation** (پیش‌پردازش و آماده‌سازی داده‌ها) 9

ب) **Encoding and Fitness Function** (کدگذاری و تابع برازندگی) 12

ج) **Genetic Algorithm Settings/Parameters** (تنظیمات الگوریتم ژنتیک) 14

د) **Visualization and Reporting** (تصویرسازی و گزارش) 19

نتیجه‌گیری 23

مقدمه

در این گزارش، پاسخ تکالیف در دو بخش مجزا تدوین شده است. در بخش اول (سوالات ۱ تا ۳)، به بررسی مفاهیم تئوری و محاسبات پایه‌ای شبکه‌های عصبی پرداختم تا درک بهتری از نحوه کارکرد این مدل‌ها و الگوریتم‌های بهینه‌سازی پیدا کنم. اما بخش اصلی و چالش‌برانگیز این تکلیف در سوال ۴ و پیاده‌سازی عملی آن قرار دارد. در این بخش، یک سیستم تشخیص سرطان پستان را با استفاده از مجموعه داده معروف Breast Cancer پیاده‌سازی کردم. ایده اصلی این بود که به جای استفاده از روش‌های رایج مبتنی بر گرادیان (مثل Backpropagation)، از الگوریتم ژنتیک (GA) برای آموزش یک شبکه عصبی پرسپترون چندلایه (MLP) استفاده کنم.

برای این کار، ابتدا داده‌ها را با روش Z-score نرمال‌سازی کردم و با نسبت ۷۰ به ۳۰ برای آموزش و تست تقسیم کردم. معماری شبکه را به صورت $[30, 5, 1]$ در نظر گرفتم که باعث شد در مجموع با ۱۶۱ وزن و بایاس سروکار داشته باشم. در این پروژه نشان داده‌ام که چطور این ۱۶۱ پارامتر به عنوان یک «کروموزوم» در الگوریتم ژنتیک کدگذاری می‌شوند و با تنظیم پارامترهایی مثل جمعیت ۵۰ تایی و ۱۰۰ نسل تکامل، شبکه به مرور یاد می‌گیرد که داده‌های مربوط به تومورهای خوش‌خیم و بدخیم را با دقت بالایی از هم تفکیک کند.

سوال اول (بخش اول)

یک نورون تنها (پرسپترون ساده) با دو ورودی (x_1 و x_2) و یک تابع فعال‌ساز پله‌ای سخت (Hard-Limit) به صورت زیر در نظر بگیرید:

$$\hat{y} = \text{hardlim}(w_1x_1 + w_2x_2 + b) = \begin{cases} 1 & w_1x_1 + w_2x_2 + b \geq 0 \\ 0 & w_1x_1 + w_2x_2 + b < 0 \end{cases}$$

می‌خواهیم از الگوریتم ژنتیک برای تنظیم وزن‌ها (w_1, w_2) و بایاس (b) این نورون استفاده کنیم تا گیت منطقی AND را پیاده‌سازی کند. هر یک از این ۳ پارامتر پیوسته، با یک رشته باینری ۴ بیتی کدگذاری شده‌اند که در مجموع یک کروموزوم ۱۲ بیتی را تشکیل می‌دهند (۴ بیت اول برای w_1، ۴ بیت دوم برای w_2 و ۴ بیت سوم برای b).

فرمول نگاشت و رمزگشایی (Decode) یک رشته باینری ۴ بیتی به عدد واقعی دسیما در بازه $[-2, 2]$ به صورت زیر تعریف شده است:

$$\text{Value} = -2 + \text{Decimal(binary)} \times \frac{2-(-2)}{2^4-1} = -2 + \text{Decimal(binary)} \times \frac{4}{15}$$

فرمول نگاشت به درستی طراحی شده است؛ زیرا حداقل مقدار یک رشته ۴ بیتی برابر $\min(decimal) = 0$ است و حداکثر مقدار هم برای یک رشته ۴ بیتی برابر $\max(decimal) = 15$ می‌باشد؛ پس مینیمم و ماکسیمم $Value$ برابرند با:

$$Value = -2 + \frac{4}{15}\, decimal = \frac{4 \times decimal - 30}{15}$$

$$(Value)_{min} = -2 \quad , \quad (Value)_{max} = +2$$

همچنین مزیت دیگری هم که این فرمول دارد، این است که به دلیل خطی بودن فرمول، داده‌ها به صورت یک تصاعد حسابی با قدر تفاضل $d = \frac{4}{15}$ پخش می‌شوند.

الف)

اگر یک کروموزوم تصادفی در جمعیت به صورت 110001011000 تولید شده باشد، مقادیر واقعی وزن‌ها (w_1, w_2) و بایاس (b) را به صورت اعشاری محاسبه کنید.

$$w_1 \propto (1100)_2 = 12$$

$$w_2 \propto (0101)_2 = 5$$

$$b \propto (1000)_2 = 8$$

حالا با فرمول $Value$ ارزش هر یک را محاسبه می‌کنیم:

$$w_1 = \frac{48-30}{15} = \frac{6}{5} \quad \rightarrow \quad w_1 = 1.200$$

$$w_2 = \frac{20-30}{15} = -\frac{2}{3} \quad \rightarrow \quad w_2 \approx -0.667$$

$$b = \frac{32-30}{15} = \frac{2}{15} \quad \rightarrow \quad b \approx 0.133$$

ب)

خروجی پیش‌بینی شده توسط این نورون ($\hat{y}$) را به ازای هر ۴ حالت ورودی گیت AND یعنی $(0,0)(0,1)(1,0)$ و $(1,1)$ با استفاده از وزن‌های به دست آمده مشخص نمایید.

تابع پله‌ای سخت ($Hard\text{-}Limit\ Step\ Function$) به صورت زیر تعریف می‌شود:

$$u_0(x) = \begin{cases} 1 & ; \quad x \geq 0 \\ 0 & ; \quad x < 0 \end{cases}$$

همچنین، گره net یا همان Z به روش زیر محاسبه می‌شود:

$$Z(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$$

با مقادیر وزن‌ها و بایاس قبلی جدول زیر را خواهیم داشت:

$index$	x_1	x_2	Z	$\hat{y} = u_0(Z)$	y_{true}
1	0	0	0.133	1	0
2	0	1	-0.534	0	0
3	1	0	1.333	1	0
4	1	1	0.666	1	1

سوال دوم (بخش اول)

برای ارزیابی کیفیت عملکرد شبکه‌های عصبی تولید شده در سوال اول، نیاز به طراحی یک تابع برازندگی (Fitness Function) داریم تا الگوریتم ژنتیک بتواند بر اساس آن فرآیند «بقای اصلح» را هدایت کند. هدف ما این است که شبکه عصبی کمترین خطای دسته‌بندی را روی جدول صحت گیت AND داشته باشد.

الف)

اگر تابع هزینه را به صورت مجموع مجذور خطاها (SSE) روی هر ۴ نمونه داده به صورت زیر تعریف کنیم:

$$E = \sum_{i=1}^{4} (y_i - \hat{y}_i)^2$$

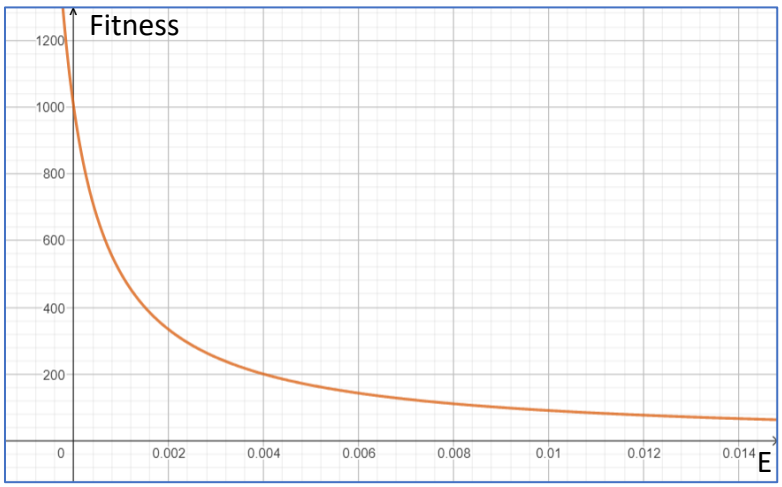
که در آن y_i خروجی واقعی گیت AND و $\hat{y}_i$ خروجی نورون است؛ یک تابع برازندگی $(Fitness)$ مناسب، همواره مثبت و بدون خطر تقسیم بر صفر طراحی کنید که با مینیمم شدن خطا $(E \to 0)$، مقدار شایستگی کروموزوم ماکزیمم شود.

چون E مجموع تعدادی مربع عدد حقیقی است، پس حتما عددی نامنفی می‌باشد. $(E \geq 0)$

برای $Fitness$ نیاز به تابعی نزولی بر حسب E داریم که هر چه خطا کمتر بود فیتنس بیشتر باشد؛ پس یکی از بهترین حالات برای این مسئله تابع زیر است:

$$Fitness = \frac{1}{E + \epsilon}$$

به طوری که ϵ عدد بسیار کوچکی است تا از تقسیم بر صفر شدن جلوگیری کند. (مثلا $\epsilon = 0.001$)



ب)

با استفاده از تابع برازندگی طراحی شده در بخش (الف)، امتیاز شایستگی نهایی کروموزوم معرفی‌شده در سوال اول (110001011000) را محاسبه کنید. آیا این کروموزوم یک راه‌حل بهینه برای گیت AND است؟ چرا؟

$$(Fitness)_{max} = \frac{1}{(E)_{min} + \epsilon} = \frac{1}{\epsilon} = \frac{1}{0.001} = 1000$$

$$E = \sum_{i=1}^{4} (y_i - \hat{y}_i)^2$$

$$E = (y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2 + (y_4 - \hat{y}_4)^2$$

$$E = (0-1)^2 + (0-0)^2 + (0-1)^2 + (1-1)^2 = 1 + 0 + 1 + 0$$

$$E = 2 \quad \rightarrow \quad Fitness = \frac{1}{2 + \epsilon} = \frac{1}{2.001} \approx 0.499$$

خیر، این کروموزوم یک راه‌حل بهینه نیست. زیرا یک راه‌حل بهینه باید خطای صفر ($E = 0$) داشته باشد که به ماکزیمم مقدار برازندگی منجر می‌شود. اما این کروموزوم دارای خطای ۲ است (۲ پیش‌بینی غلط و ۲ پیش‌بینی درست) و برازندگی آن ($Fitness = 0.499$) بسیار کمتر از حالت بهینه و ماکسیمم ($Fitness = 1000$) است.

سوال سوم (بخش اول)

دو کروموزوم والد ۸ بیتی را در نظر بگیرید که هر کدام نشان‌دهنده وزن‌های w_1 و w_2 یک شبکه عصبی بدون بایاس هستند (۴ بیت سمت چپ برای w_1 و ۴ بیت سمت راست برای w_2). بازه نگاشت ژن‌ها به فضای پیوسته همان بازه $[-2, 2]$ سوال اول است.

$$P_1 = [1101 \mid 0011], \quad P_2 = [0110 \mid 1100]$$

الف)

عملگر ترکیب تک‌نقطه‌ای (One-Point Crossover) را دقیقاً از مرز بین دو وزن (بعد از بیت چهارم) اعمال کنید و فرزندان حاصل (C_1 و C_2) را بنویسید. از دیدگاه ساختار عملکردی شبکه عصبی توضیح دهید که این نقطه برش خاص چه تأثیری بر روی معماری فرزندان می‌گذارد؟

$$P_1 = [P_{1,1} \mid P_{1,2}] = [1101 \mid 0011] \quad \rightarrow \quad (w_1 \propto 13 \quad , \quad w_2 \propto 3)$$

$$P_2 = [P_{2,1} \mid P_{2,2}] = [0110 \mid 1100] \quad \rightarrow \quad (w_1 \propto 6 \quad , \quad w_2 \propto 12)$$

در نتیجه خواهیم داشت:

$$C_1 = [P_{1,1} \mid P_{2,2}] = [1101 \mid 1100] \quad \rightarrow \quad (w_1 \propto 13 \quad , \quad w_2 \propto 12)$$

$$C_2 = [P_{2,1} \mid P_{1,2}] = [0110 \mid 0011] \quad \rightarrow \quad (w_1 \propto 6 \quad , \quad w_2 \propto 3)$$

تاثیر برش از مرز بین دو وزن بر روی معماری فرزندان:

از دیدگاه عملکردی شبکه عصبی، وقتی نقطه برش دقیقاً در مرز بین دو ژن (وزن‌های w_1 و w_2) قرار می‌گیرد، به این معنی است که **بلوک‌های سازنده هر وزن دست‌نخورده باقی می‌مانند.**

فرزند C_1 دقیقاً وزن w_1 را از والد اول و وزن w_2 را از والد دوم به ارث می‌برد (و برعکس برای C_2). این نوع تقاطع باعث می‌شود مقادیر عددی وزن‌هایی که در والدین احتمالاً مقادیر خوبی بوده‌اند، بدون تخریب کد باینری‌شان، به فرزندان منتقل و با هم ترکیب شوند. اگر برش در وسط بیت‌های یک وزن اتفاق می‌افتاد، مقدار آن وزن به طور تصادفی و غیرقابل پیش‌بینی تغییر می‌کرد.

ب)

اگر روی فرزند اول (C_1) عملگر جهش (Bit-Flip Mutation) رخ دهد و آخرین بیت آن (راست‌ترین بیت) معکوس شود، مقدار عددی وزن دوم (w_2) در این شبکه عصبی چقدر و در چه جهتی (کاهش یا افزایش) تغییر می‌کند؟ تحلیل کنید که جهش در بیت‌های کم‌ارزش (LSB) در مقایسه با بیت‌های پرارزش (MSB) چه تفاوتی در رفتار و پایداری شبکه عصبی ایجاد می‌کند.

فرمولی که برای $Value$ داشتیم به شرح زیر بود (هنوز هم بازه نگاشت $[-2,2]$ است.):

$$Value = -2 + \frac{4}{15}\, decimal = \frac{4 \times decimal - 30}{15}$$

در بخش قبلی داشتیم:

$$C_1 = [P_{1,1} \mid P_{2,2}] = [1101 \mid 1100]$$

$$\rightarrow (w_1 = \frac{52-30}{15} = 1.467 \quad , \quad w_2 = \frac{48-30}{15} = 1.200)$$

پس از Mutation خواهیم داشت:

$$(C_1)_{mutated} = [1101 \mid 1101]$$

$$\rightarrow (w_1 = \frac{52-30}{15} = 1.467 \quad , \quad w_2 = \frac{52-30}{15} = 1.467)$$

$$\Delta w_2 = 1.467 - 1.200 = 0.267$$

در این حالت w_1 **دست نخورده** باقی می‌ماند و w_2 **به اندازه 0.267 واحد افزایش** می‌یابد.

تحلیل جهش LSB در مقایسه با MSB:

- **جهش در LSB (کم‌ارزش‌ترین بیت):** کمترین تغییر ممکن (به اندازه یک گام Δ) را در مقدار وزن ایجاد می‌کند. این نوع جهش باعث تغییرات بسیار جزئی در خروجی شبکه عصبی می‌شود و برای **جستجوی محلی (Local Search) و تنظیم دقیق (Fine-tuning)** وزن‌ها مفید است و پایداری شبکه را حفظ می‌کند.
- **جهش در MSB (پرارزش‌ترین بیت):** باعث بزرگترین تغییر ممکن در مقدار وزن می‌شود (نصف طول بازه). این تغییر ناگهانی می‌تواند رفتار شبکه را به شدت تغییر دهد و آن را ناپایدار کند، اما برای **جستجوی سراسری (Global Search) و خروج از بهینه‌های محلی** بسیار موثر است.

سوال چهارم (بخش دوم)

سوال چهارم: آموزش تکاملی شبکه عصبی (Neuroevolution) روی دیتاست سرطان سینه

در روش‌های سنتی، شبکه‌های عصبی با الگوریتم‌های مبتنی بر گرادیان (مانند پس‌انتشار خطا یا Backpropagation) آموزش می‌بینند. در این سوال می‌خواهیم از **الگوریتم ژنتیک** به عنوان یک بهینه‌ساز سراسری برای پیدا کردن وزن‌ها و بایاس‌های یک شبکه عصبی استفاده کنیم.

فرض کنید یک شبکه عصبی پرسپترون چندلایه (MLP) با ساختار $[30, 5, 1]$ دارید؛ یعنی ۳۰ نورون در لایه ورودی (منطبق بر ۳۰ ویژگی دیتاست)، ۵ نورون در لایه پنهان و ۱ نورون در لایه خروجی با تابع فعال‌ساز سیگموئید (برای پیش‌بینی دوتایی خوش‌خیم یا بدخیم بودن تومور).

با استفاده از زبان برنامه‌نویسی پایتون و کتابخانه‌های scikit-learn و PyGAD (یا پیاده‌سازی دستی خود)، برنامه‌ای بنویسید که فرآیند تکامل این شبکه را شبیه‌سازی کند.

من با استفاده از پیاده‌سازی دستی و فقط با استفاده از کتابخانه Numpy این شبکه را می‌سازم.

البته برای خواندن فایل csv داده‌ها از Pandas و برای Plotting از Matplotlib هم استفاده می‌کنیم.

الف) Data Preprocessing and Preparation (پیش‌پردازش و آماده‌سازی داده‌ها)

پیش‌پردازش و آماده‌سازی داده‌ها: دیتاست Breast Cancer را بارگذاری کرده و داده‌ها را به صورت استاندارد (Z-score) نرمال‌سازی کنید. سپس داده‌ها را به نسبت ۷۰ درصد برای آموزش (Train) و ۳۰ درصد برای تست (Test) تقسیم نمایید.

چون فایل csv را نداریم، ابتدا باید آن را دانلود کنیم؛ فقط برای بخش دانلود چاره‌ای نداریم، جز اینکه از کتابخانه‌های $sklearn.datasets$ و os استفاده کنیم و سپس تماما آن را با $numpy$ پیاده‌سازی می‌کنیم:

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# just for loading dataset:
import os
from sklearn.datasets import load_breast_cancer

if not os.path.exists("breast_cancer.csv"):
    data = load_breast_cancer()
    df = pd.DataFrame(data.data , columns = data.feature_names)
    df['target'] = data.target
    df.to_csv("breast_cancer.csv" , index = False)
```

حالا که دیتاست ما درون فایل breast_cancer.csv آماده است، کار خود را شروع می‌کنیم.

اگر به این فایل csv دقت کنیم، این دیتاست شامل ۵۶۹ نمونه (samples) در سطرها و ۳۰ ویژگی (feature) در ستون‌ها است که ۷۰ درصد نمونه‌ها (یعنی ۳۹۹ نمونه) برای آموزش (Train) و ۳۰ درصد بقیه این نمونه‌ها (یعنی ۱۷۰ نمونه) برای ارزیابی (Test) استفاده می‌شوند.

برای نرمال‌سازی هم چون Y همان label های خروجی (0 و 1) است و اعداد صفر و یک برای شبکه عصبی مناسب هستند، نیازی به نرمال‌سازی خروجی نداریم؛ اما ورودی ها باید نرمال شوند.

یعنی به شکل زیر داده‌های آموزشی و تست خود را می‌سازیم:

```
143  #-------------------------------------------------------------------------------------#
144  ### 1. Data Preprocessing and Preparation:
145  df = pd.read_csv("breast_cancer.csv")
146  df = df.sample(frac=1, random_state=42).reset_index(drop=True) # Shuffling dataset
147  N_samples , N_columns = df.shape
148
149  N_Test = (30 * N_samples) // 100
150  N_Train = N_samples - N_Test
151  N_features = N_columns - 1
152
153  X_TRAIN = df.iloc[:N_Train , :-1].to_numpy()
154  X_TEST = df.iloc[N_Train: , :-1].to_numpy()
155
156  mean = np.mean(X_TRAIN, axis=0)
157  std = np.std(X_TRAIN, axis=0)
158
159  X_TRAIN_SCALED = (X_TRAIN - mean) / (std + Epsilon)
160  X_TEST_SCALED = (X_TEST - mean) / (std + Epsilon)
161  Y_TRAIN = df.iloc[:N_Train , -1].to_numpy()
162  Y_TEST = df.iloc[N_Train: , -1].to_numpy()
```

در اینجا $Epsilon = 10^{-8}$ به عنوان عدد کوچکی انتخاب شده که از تقسیم بر صفر شدن جلوگیری کند.

میانگین و انحراف معیار داده‌های Train را محاسبه می‌کنیم و سپس با آن فرمول $Z\text{-}Score$ را اجرا می‌کنیم.

$$X_{norm} = \frac{X - \mu}{\sigma}$$

همچنین برای سهولت روند آموزش، داده‌های زیر را به عنوان اعداد ثابت انتخاب کرده‌ایم:

```
#-------------------------------------------------------------------------------#
### Constants:
Epsilon = 1e-8
Population = 50
Generations = 100
Crossover_rate = 0.80
Mutation_rate = 0.05
Tournament_size = 3
Pairs = Population // 2
```

در این پیاده‌سازی از مکانیزم نخبه‌گرایی (Elitism) استفاده نشده است.

دلیل این امر آن است که استفاده از نخبه‌گرایی می‌تواند منجر به همگرایی بسیار سریع الگوریتم (و در مواردی همگرایی زودهنگام) شود. عدم استفاده از این مکانیزم به ما اجازه می‌دهد تا روند تکامل و تغییرات تدریجی تابع برازندگی (Fitness) در طول نسل‌های مختلف را با وضوح بیشتری مشاهده و تحلیل کنیم.

همچنین کروموزوم‌های خود را در یک فایل NumPy-zip یا (npz) ذخیره می‌کنیم تا بعدا هم این داده‌ها را داشته باشیم و موجود باشند.

همچنین برای مقادیر اولیه کروموزوم‌ها و جلوگیری از دو مشکل بزرگ انفجار گرادیان (Exploding Gradient) و محو گرادیان (Vanishing Gradient)، از He Initialization استفاده می‌کنیم؛ به طور که مقادیر اولیه وزن‌ها به صورت توزیع نرمال/گوسی با میانگین صفر و واریانس $\frac{2}{n_{in}}$ انتخاب می‌شوند:

```
#-------------------------------------------------------------------------------#
# He initialization:
chs = np.zeros((Population , 161))
# W1:
var1 = 2 / 30
std1 = np.sqrt(var1)
chs[: , :150] = std1 * np.random.randn(Population , 150)
# W2:
var2 = 2 / 5
std2 = np.sqrt(var2)
chs[: , 150:155] = std2 * np.random.randn(Population , 5)
# Saving:
np.savez("chromosomes.npz" , chromosomes = chs)

FILE = np.load("chromosomes.npz")
CHR = FILE['chromosomes']
```

ب) Encoding and Fitness Function (کدگذاری و تابع برازندگی)

۱. محاسبه طول کروموزوم (تعداد کل وزن‌ها و بایاس‌ها):

ما یک شبکه با ساختار $[30 , 5 , 1]$ داریم. هر کروموزوم در الگوریتم ژنتیک باید تمام این وزن‌ها و بایاس‌ها را در خود جای دهد (به صورت یک آرایه یک‌بعدی). محاسبه آن به این شکل است:

- **بین لایه ورودی و مخفی:** 30 ورودی به 5 نورون متصل می‌شوند. پس $30 \times 5 = 150$ وزن و 5 بایاس (برای هر نورون مخفی یک بایاس) داریم. (جمعاً 155 پارامتر)
- **بین لایه مخفی و خروجی:** 5 خروجیِ لایه مخفی به 1 نورون خروجی متصل می‌شوند. پس در واقع $5 \times 1 = 5$ وزن و 1 بایاس داریم. (جمعاً 6 پارامتر)
- **طول کل کروموزوم:** $155 + 6 = 161$. بنابراین هر کروموزوم ما یک آرایه به طول 161 است.

```
163  #-------------------------------------------------------------------------------#
164  ### 2. Encoding and Fitness Function:
165  neurons = [30 , 5 , 1]
166
167  C_W1 = neurons[1] * neurons[0]   # (5 , 30)
168  C_W2 = neurons[2] * neurons[1]   # (1 , 5)
169  C_b1 = neurons[1] * 1            # (5 , 1)
170  C_b2 = neurons[2] * 1            # (1 , 1)
171
172  C_tot = C_W1 + C_W2 + C_b1 + C_b2
173  print(f"The lengths of all the chromosomes: {C_tot}\n") # 161
```

خروجی به شکل زیر خواهد بود:

```
The lengths of all the chromosomes: 161
```

(نکته: در پیاده‌سازی این پروژه، برای محاسبه تابع برازندگی (Fitness Function) در طول اجرای الگوریتم ژنتیک (مرحله بهینه‌سازی وزن‌ها)، منحصراً از داده‌های آموزش (Train Data) استفاده شده است. در صورتی که از داده‌های ارزیابی (Test Data) در فرآیند آموزش و انتخاب کروموزوم‌ها استفاده شود، پدیده نشت داده (Data Leakage) رخ می‌دهد. در این حالت، مدل اطلاعات مجموعه ارزیابی را در حین آموزش می‌بیند که منجر به بیش‌برازش (Overfitting) روی داده‌های تست شده و نتایج ارزیابی نهایی را کاملاً فاقد اعتبار علمی می‌سازد.

بنابراین، به منظور رعایت اصول صحیح یادگیری ماشین و سنجش واقعی قدرت تعمیم‌پذیری (Generalization) شبکه عصبی، مجموعه داده‌های تست به طور کامل از حلقه آموزش الگوریتم ژنتیک ایزوله نگه داشته شدند و صرفاً پس از اتمام تمامی نسل‌ها، برای محاسبه معیارهای ارزیابی نهایی (از جمله Accuracy، ماتریس اغتشاش، Precision و Recall) مورد استفاده قرار گرفتند.

پس تابع فیتنس را به شکل زیر تعریف می‌کنیم:

```
38  #-------------------------------------------------------------------------------#
39  ### Definitions of Functions:
40
41  #|=============================================|#
42  #| 1. Calculating Fitnesses
43  #|=============================================|#
44  def Fitness(Y_true , Y_pred):
45      Accuracy = np.mean(Y_true == Y_pred, axis=1) * 100
46      return Accuracy
47
```

مزیت دیگری هم که این پیاده‌سازی دارد، این است که تابع Fitness ما همان Accuracy ما هم هست و در واقع Fitness محدود به بازه $[0,100]$ شده است که مقایسه بین کروموزوم‌ها را راحت‌تر می‌کند.

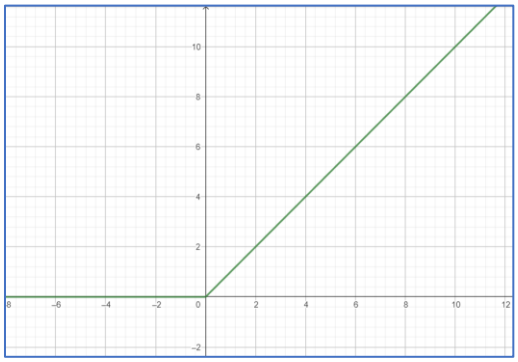
ج) Genetic Algorithm Settings/Parameters (تنظیمات الگوریتم ژنتیک)

حالا همه چیز آماده شده تا الگوریتم ژنتیک را پیاده‌سازی کنیم؛

ما در اینجا دو تابع فعال‌ساز برای مسیر رفت (*feed-forward*) نیاز داریم:

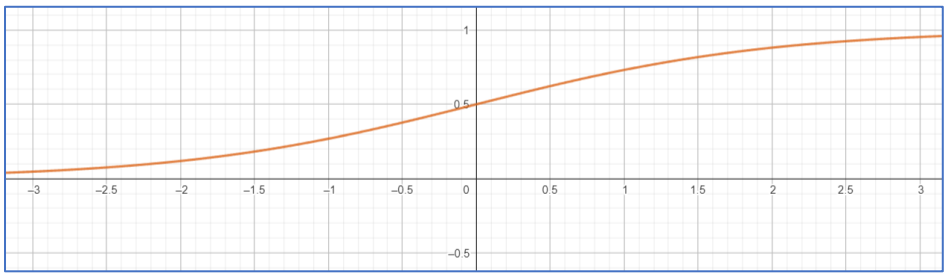
1- ReLU

```
48  #|=============================================|#
49  #| 2. ReLU Activation Function
50  #|=============================================|#
51  def ReLU(x):
52      return np.maximum(0, x)
53
```



2- Sigmoid

```
54  #|=============================================|#
55  #| 3. Sigmoid Activation Function
56  #|=============================================|#
57  def Sigmoid(x):
58      x = np.clip(x, -500, 500)
59      return 1 / (1 + np.exp(-x))
60
```



حالا یک تابع کمکی می‌نویسیم که رشته یک بعدی کروموزوم را به وزن ها تبدیل کند؛ یعنی از ۱۶۱ وزن داخل آن، ۱۵۰ وزن اول را به w_1 و ۵ وزن بعدی را به w_2 و دوباره ۵ وزن بعدی را به b_1 و در آخر یک وزن باقی‌مانده را به b_2 اختصاص دهد و این کار را برای کل Population انجام دهد:

```
#|=============================================|#
#| 4. Making Weights from Chromosomes
#|=============================================|#
def Chromosomes_to_Weights(Chs):
    w1 = Chs[: , :150].reshape(-1,5,30)
    w2 = Chs[: , 150:155].reshape(-1,1,5)
    b1 = Chs[: , 155:160].reshape(-1,5,1)
    b2 = Chs[: , 160].reshape(-1,1,1)
    return w1 , w2 , b1 , b2

```

حالا به راحتی با کمک این تابع، تابع $feed\text{-}forward$ را می‌سازیم:

```
#|=============================================|#
#| 5. Forward Path
#|=============================================|#
def feed_forward(Chs , X):
    w1 , w2 , b1 , b2 = Chromosomes_to_Weights(Chs)
    z1 = (w1 @ X.T) + b1
    a1 = ReLU(z1)
    z2 = (w2 @ a1) + b2
    a2 = Sigmoid(z2)
    y = np.where(a2 >= 0.5 , 1 , 0)
    return np.squeeze(y , axis=1)

```

در اینجا از فرمول‌های MLP برای محاسبه خروجی هر لایه استفاده کرده‌ایم. $(f(.) = Activation\ Function)$

$$z = Wx + b \quad , \quad a = f(z)$$

ابتدا از تابع کمکی استفاده کردیم تا وزن‌ها را به دست آوریم؛ تمام مقادیر خروجی بزرگتر مساوی 0.5 در شبکه را 1 (تخمین بدخیم بودن سرطان) و تمام مقادیر خروجی کوچکتر از 0.5 در شبکه را 0 (تخمین خوش‌خیم بودن سرطان) تفسیر می‌کنیم.

ابعاد نهایی به صورت $(Population\ ,1\ ,\ N_{samples})$ هست که برای از بین رفتن بعد دوم (axis=1) که یک هست از تابع $np.squeeze$ استفاده می‌کنیم.

حالا تابع *Tournament* را می‌سازیم که دو دسته ۳ تایی انتخاب می‌کند و در آن دست کسی که بالاترین *fitness* را دارد برنده است و از بین جمعیت دو نفر به عنوان والد اول (P_1) و والد دوم (P_2) انتخاب می‌شوند. بعد عمل *Crossover* را با روش *one-point* روی آن دو با احتمال *80%* انجام می‌دهیم و دو فرزند C_1 و C_2 ایجاد می‌کنیم. اگر *Crossover* رخ نداد، دو والد عینا به نسل بعد منتقل می‌شوند:

```
#|=============================================|#
#| 6. Conducting the Tournaments
#|=============================================|#
def Tournament(Chs, fitness, k=Tournament_size):
    pop = fitness.shape[0]
    # Parent 1:
    arr = []
    idx = np.random.randint(0 , pop-1 , size=k).tolist()
    for index in idx:
        arr.append( (index , fitness[index]) )
    arr_sorted = sorted(arr , key=lambda x : x[1] , reverse=True)
    p1 = arr_sorted[0][0]
    # Parent 2:
    arr = []
    idx = np.random.randint(0 , pop-1 , size=k).tolist()
    for index in idx:
        arr.append( (index , fitness[index]) )
    arr_sorted = sorted(arr , key=lambda x : x[1] , reverse=True)
    p2 = arr_sorted[0][0]
    P1 = Chs[p1 , :]
    P2 = Chs[p2 , :]
    # Crossover Rate:
    cr = np.random.rand()
    if cr < Crossover_rate:
        # Making Children:
        split_point = int(np.random.randint(0 , 160))
        C1 = np.zeros_like(P1)
        C1[:split_point] = P1[:split_point]
        C1[split_point:] = P2[split_point:]
        C2 = np.zeros_like(P2)
        C2[:split_point] = P2[:split_point]
        C2[split_point:] = P1[split_point:]
    else:
        C1 = P1
        C2 = P2
    return C1 , C2
```

حالا تابعی به نام *Crossover* می‌سازیم که تا جایی که لازم باشد ترکیب را انجام می‌دهد و جمعیت جدید را با این فرزندان ترکیبی از والدین پر کند:

```
#|=============================================|#
#| 7. Performing Crossover on the Population
#|=============================================|#
def Crossover(Chs, fitness):
    New_Chs = np.zeros_like(Chs)
    C = 0
    for _ in range(Pairs):
        child1 , child2 = Tournament(Chs, fitness)
        New_Chs[C , :] = child1
        C += 1
        New_Chs[C , :] = child2
        C += 1
    return New_Chs

```

در آخر هم تابع Mutation را می‌سازیم که با احتمال 5% یک درایه را با یک نویز گوسی جمع می‌کند تا فضای جستجو گسترده‌تر شود و در مینیمم محلی گیر نکنیم:

```
#|=============================================|#
#| 8. Performing Mutation on the Population
#|=============================================|#
def Mutation(Chs , Rate=Mutation_rate):
    Mask = np.random.random(Chs.shape)
    Mask = np.where(Mask < Rate , 1 , 0)
    Noise = np.random.randn(Chs.shape[0] , Chs.shape[1])
    Chs += Noise * Mask
    return Chs
```

حالا از تمام این توابع در مراحل و حلقه آموزش استفاده می‌کنیم:

```
174  #-------------------------------------------------------------------------------------#
175  ### 3. Genetic Algorithm Settings/Parameters:
176  Max_Fitnesses = []
177  Mean_Fitnesses = []
178  best_index = None
179
180  for gen in range(Generations):
181      Y_pred = feed_forward(CHR , X_TRAIN_SCALED)
182      Gen_fitness = Fitness(Y_TRAIN , Y_pred)
183      best_index = int(np.argmax(Gen_fitness))
184      best_chromosome = CHR[best_index , :].copy()
185
186      CHR = Crossover(CHR , Gen_fitness)
187      CHR = Mutation(CHR)
188      Max_Fitnesses.append( float(np.max(Gen_fitness)) )
189      Mean_Fitnesses.append( float(np.mean(Gen_fitness)) )
190
191  np.savez("chromosomes.npz" , chromosomes = CHR)
```

در هر نسل بهترین فیتنس ($Max\ Fitnesses$) و میانگین فیتنس‌ها ($Mean\ Fitnesses$) را ذخیره می‌کنیم تا بتوانیم آن‌ها را رسم و مصورسازی کنیم.

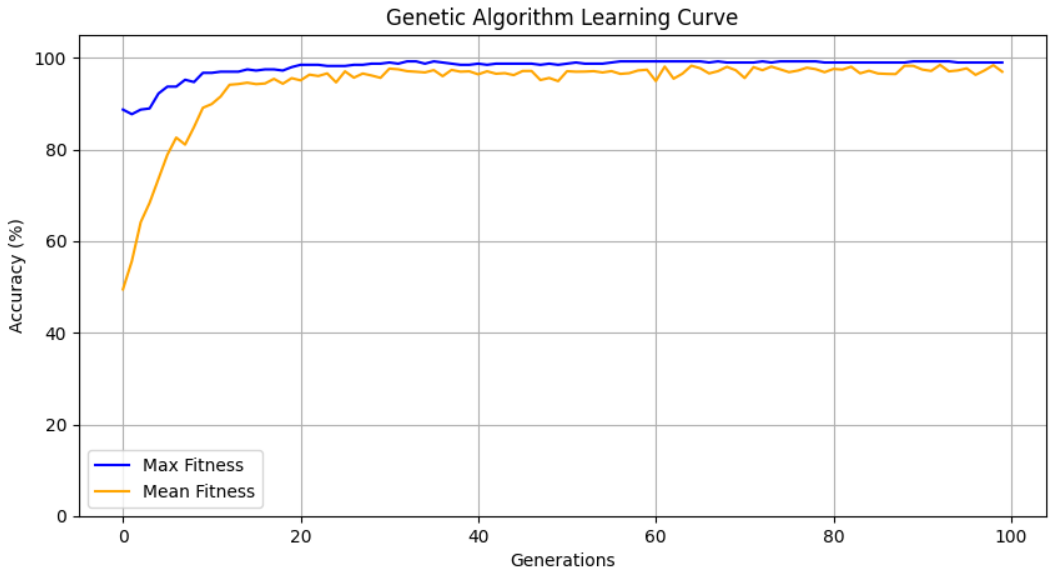
بهترین کروموزوم (کروموزومی که بهترین فیتنس را در بین تمام نسل‌ها گرفته است) را هم در هر نسل درون متغیر $best\ chromosome$ ذخیره می‌کنیم که برای بخش بعدی آن را داشته باشیم.

در آخر هم کل آرایه کروموزوم‌ها را درون فایل $chromosomes.npz$ ذخیره می‌کنیم.

د) Visualization and Reporting (تصویرسازی و گزارش)

```
192  #-------------------------------------------------------------------------------------#
193  ### 4. Visualization and Reporting:
194  plt.figure(figsize=(10,5))
195  plt.plot(Max_Fitnesses, label='Max Fitness', color='blue')
196  plt.plot(Mean_Fitnesses, label='Mean Fitness', color='orange')
197  plt.title(f'Genetic Algorithm Learning Curve')
198  plt.xlabel('Generations')
199  plt.ylabel('Accuracy (%)')
200  plt.ylim([0 , 105])
201  plt.legend()
202  plt.grid(True)
203  plt.show()
204
```

خروجی نمودار تغییرات فیتنس به شرح زیر است:



می‌بینیم که تقریبا از نسل ۲۰ به بعد $Max\ Fitness$ ما به شدت به دقت ۱۰۰ درصد نزدیک شده است. حالا $best\ chromosome$ را روی کل داده‌های تست می‌سنجیم تا Accuracy آن را روی داده‌هایی که تا به حال آموزش ندیده بسنجیم:

```
205  Y_pred_test = feed_forward(best_chromosome.reshape(1,-1) , X_TEST_SCALED)
206  accuracy = Fitness(Y_TEST , Y_pred_test)
207  print(f"Test Accuracy: {accuracy[0]:.2f}%\n")
```

نتیجه به شکل زیر خواهد بود:

```
Test Accuracy: 96.47%
```

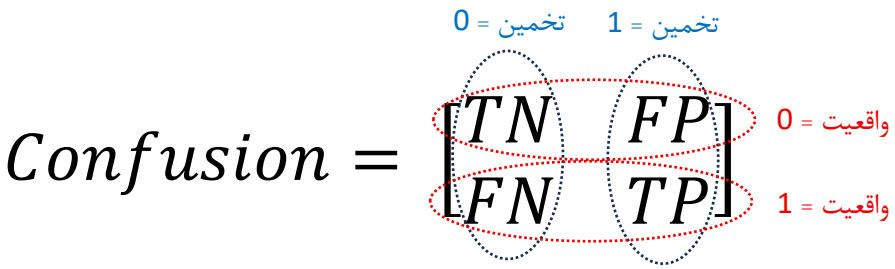
حالا ماتریس اغتشاش و پارامتر دقت و پارامتر فراخوانی را تعریف می‌کنیم:

۱. ماتریس درهم‌ریختگی یا اغتشاش (Confusion Matrix)

این ماتریس یک جدول است که عملکرد الگوریتم طبقه‌بندی را با مقایسه مقادیر پیش‌بینی‌شده توسط مدل با مقادیر واقعی نشان می‌دهد. در مسئله ما که دو کلاس خوش‌خیم و بدخیم وجود دارند، این ماتریس ۴ بخش دارد:

- **منفی صحیح (TN یا True Negative)**: فرد سالم بوده و مدل هم به درستی او را سالم تشخیص داده است. (واقعیت=0 / تخمین=0)
- **مثبت کاذب (FP یا False Positive)**: فرد سالم بوده، اما مدل به اشتباه او را بیمار تشخیص داده است (هشدار اشتباه یا خطای نوع اول). (واقعیت=0 / تخمین=1)
- **منفی کاذب (FN یا False Negative)**: فرد بیمار بوده، اما مدل به اشتباه او را سالم تشخیص داده است (بیماری که از قلم افتاده یا خطای نوع دوم). (واقعیت=1 / تخمین=0)
- **مثبت صحیح (TP یا True Positive)**: بیمار واقعاً مبتلا به سرطان بوده و مدل هم به درستی بیماری را تشخیص داده است. (واقعیت=1 / تخمین=1)

این ماتریس، یک ماتریس ۲ × ۲ هست که به شکل زیر تعریف می‌شود:



(نکته: صفر به معنی خوش‌خیم بودن و یک به معنی بدخیم بودن است.)

۲. معیار دقت یا صحت (Precision)

دقت نشان می‌دهد که از بین تمام مواردی که الگوریتم به عنوان «مثبت» (بیمار) پیش‌بینی کرده است، واقعاً چه تعداد از آن‌ها بیمار بوده‌اند. در واقع این معیار می‌گوید: *«اگر مدل گفت این فرد بیمار است، چقدر می‌توان به حرف او اعتماد کرد؟»*

$$Precision = \frac{TP}{TP + FP}$$

بالا بودن این معیار به معنای کم بودن مثبت‌های کاذب (FP) است.

۳. معیار فراخوانی، بازخوانی یا حساسیت (Recall / Sensitivity)

فراخوانی نشان می‌دهد که از بین کل افرادی که در واقعیت بیمار بوده‌اند، مدل موفق شده چند درصد آن‌ها را پیدا کند. در واقع این معیار می‌گوید: *«مدل چه نسبتی از بیماران واقعی را توانست کشف کند؟»*

$$Recall = \frac{TP}{TP + FN}$$

در مسائل پزشکی مثل تشخیص سرطان پستان، معیار **Recall** معمولاً اهمیت بسیار بیشتری دارد؛ زیرا ترجیح می‌دهیم یک فرد سالم به اشتباه بیمار تشخیص داده شود (و با آزمایش‌های بیشتر متوجه سلامتی او شویم)، اما هیچ بیمار سرطانی به عنوان سالم تشخیص داده نشود و بدون درمان رها نشود (اهمیت کاهش FN).

حالا کدهای این مفاهیم را به شکل زیر می‌زنیم:

```
Y_pred_test_flat = Y_pred_test.flatten()

TP = np.sum((Y_TEST == 1) & (Y_pred_test_flat == 1))
TN = np.sum((Y_TEST == 0) & (Y_pred_test_flat == 0))
FP = np.sum((Y_TEST == 0) & (Y_pred_test_flat == 1))
FN = np.sum((Y_TEST == 1) & (Y_pred_test_flat == 0))

print(f"True Positives (TP): {TP}")
print(f"True Negatives (TN): {TN}")
print(f"False Positives (FP): {FP}")
print(f"False Negatives (FN): {FN}")

Confusion_Matrix = np.array([
    [TN , FP],
    [FN , TP]
])
Precision = (100 * TP) / (TP + FP)
Recall = (100 * TP) / (TP + FN)

print(f"\nConfusion Matrix:\n{Confusion_Matrix}")
print(f"Precision: {Precision:.2f}%")
print(f"Recall: {Recall:.2f}%\n")
```

خروجی به این صورت خواهد بود:

```
True Positives (TP): 98
True Negatives (TN): 66
False Positives (FP): 6
False Negatives (FN): 0

Confusion Matrix:
[[66  6]
 [ 0 98]]
Precision: 94.23%
Recall: 100.00%
```

این کروموزوم برتر روی داده‌هایی که تا به حال ندیده (TEST) دقت 94.23% و فراخوانی 100% دارد!

این یعنی مدل به خوبی مسئله را درک کرده و Overfit نشده و آماده اجرا روی داده‌های دنیای واقعی است.

نتیجه‌گیری

در نهایت، انجام این پروژه عملی در کنار سوالات تئوری نشان داد که الگوریتم ژنتیک با وجود ماهیت تصادفی‌اش، چقدر می‌تواند در بهینه‌سازی پارامترهای پیچیده مثل وزن‌های یک شبکه عصبی قدرتمند و کارآمد عمل کند. نمودار همگرایی که در انتهای گزارش رسم شده، به خوبی نشان می‌دهد که چطور مقدار خطا (*Loss*) در طول نسل‌های مختلف کاهش پیدا کرده و شبکه به سمت یک جواب بهینه هدایت شده است.

در طول پیاده‌سازی، سعی کردم به اصول علمی کار دقت ویژه داشته باشم. به عنوان مثال، برای جلوگیری از مشکل نشت داده (*Data Leakage*) که می‌تواند نتایج را به صورت کاذب خوب نشان دهد، تابع برازندگی (*Fitness*) را اکیداً فقط روی داده‌های آموزش محاسبه کردم.

همچنین برای اینکه بتوانم روند طبیعی جستجوی الگوریتم و توانایی آن در فرار از بهینه‌های محلی را دقیق‌تر ارزیابی کنم، از روش نخبه‌گرایی (*Elitism*) استفاده نکردم. خروجی‌های نهایی روی داده‌های تست (که شبکه قبلاً آن‌ها را ندیده بود) شامل ماتریس اغتشاش (*Confusion Matrix*) و معیارهای مهمی مثل $Precision$ و $Recall$ ، تایید می‌کند که مدل نهایی قابلیت اطمینان بالایی دارد و توانسته الگوهای پنهان در داده‌های پزشکی را به درستی استخراج و دسته‌بندی کند.